Abstract

This project is about building a Virtual Exam System with Built-in Anti-Cheat. It is a web application where teachers can create and schedule multiple-choice exams and students can take them online from a browser. Marks are calculated automatically, so the result is ready as soon as the student submits.

The main idea of the project is the anti-cheat part. While an exam is running, the system keeps a record of things like switching tabs, leaving fullscreen mode, copying text and, if the student allows it, webcam snapshots. Each event is saved with its time. If a student goes over the allowed number of warnings, the exam is submitted automatically. Later, the teacher can open a report for each student and see exactly what happened. The system will be developed using HTML, CSS, JavaScript, Python (Flask) and SQLite.

Introduction

Many universities now take a part of their exams online. This became very common during the COVID-19 period, and some departments still use online quizzes and class tests because they save paper and marking time. The trouble is that when students sit for the exam on their own computers, nobody is watching them. A student can open another tab, search the question on Google or message a friend, and the teacher would never know.

Tools like Google Forms are easy to use, but they were not made to stop this kind of behaviour. Paid proctoring software does try to, but it costs money and many students are uncomfortable being recorded all the time. So this project looks at a middle option: a simple exam system that a department can run by itself, which watches for the common signs of cheating and gives the teacher some proof instead of guesses.

This proposal explains the problem, the objectives, the features of the system and the way it will be built. It should be said clearly that no browser-based system can stop cheating completely. The aim here is to make cheating harder and easier to catch.

Problem Statement

When exams are held online without a supervisor, there are a few problems that teachers face again and again. Students can leave the exam page and look for answers elsewhere. If every student gets the same questions in the same order, answers are easily shared in a group chat. Some systems count the time only in the browser, so a student can refresh the page or change the timer and get extra minutes. And even when a teacher feels that someone cheated, there is usually nothing to show, because the system never recorded what happened during the exam.

Commercial proctoring tools solve some of this, but they are expensive for a university department and they need extra software. Because of all this, there is a need for a simple, low-cost exam system that can run timed exams, grade them automatically, watch the student's behaviour during the test and give the teacher a clear report afterwards.

Objectives

The main objective is to develop a web-based online exam system that has anti-cheat features built into it. The specific objectives are:

1. To create a login system with three roles: Admin, Teacher and Student.
2. To let teachers create, edit and schedule exams with multiple-choice questions.

3. To run timed exams with automatic grading and instant results.
4. To detect and record tab switching, loss of window focus, exit from fullscreen and copy-paste attempts.
5. To show questions and options in a different order for each student.
6. To check the exam time on the server so that the timer cannot be changed from the browser.
7. To submit the exam automatically when the student crosses the violation limit.
8. To take webcam snapshots and detect missing or multiple faces (optional module).
9. To give teachers a report for each attempt with a violation list, a risk score and snapshots.
10. To block a second login for a student who already has an active session.
11. To let students view their exam history and export results to CSV.

Literature Review

Online exams are currently done through three kinds of tools. The first kind is simple form and quiz tools such as Google Forms. They are free and quick to set up, and they can mark answers automatically, but they do not have much control over the exam itself. They do not shuffle per student in a proper way, do not check what the student does on the screen, and cannot lock the browser unless the device is managed.

The second kind is learning management systems like Moodle. The Moodle quiz module has question banks, time limits, shuffling and detailed grading. It is a good tool, but it is a big system that needs a server and someone to manage it. For stronger anti-cheat checking, it usually depends on extra plugins or a separate lockdown browser.

The third kind is commercial proctoring services such as Proctorio and Examus. These watch the student through webcam, microphone and screen and flag suspicious behaviour. They are powerful, but they are paid, they often need an extension or an app to be installed, and there have been complaints about privacy and wrongly flagged students.

Modern browsers also give some useful tools for this. The Page Visibility API tells a page when it is hidden, the Fullscreen API tells when the user leaves fullscreen, and getUserMedia gives access to the camera. Libraries like face-api.js can detect faces inside the browser itself. A short comparison is given below.

Feature	**Google Forms**	**Moodle Quiz**	**Paid proctoring**	**This project**
Server-checked timer	Limited	Yes	Yes	Yes
Shuffle per student	Basic	Yes	Depends	Yes
Tab-switch detection	No	Plugin needed	Yes	Yes
Webcam snapshots	No	Plugin needed	Yes	Yes (optional)
Teacher cheat report	No	Limited	Yes	Yes
Cost	Free	Free, needs server	Paid	Free

The table is based on the general features of these tools and it can differ by version or plugin.

From this it can be seen that the easy tools do not watch the student, while the strong tools are either heavy to set up or costly. The proposed system tries to fill this gap with a small system

that keeps the basic exam features and adds server-side checking and browser-based monitoring.

Proposed System

The proposed system is a web application with separate pages for the admin, teachers and students. The admin manages users and roles and can see all exams and results. A teacher creates exams, adds questions, sets the duration, schedule and violation limit, and later checks the results and the cheat reports. A student logs in, sees the exams that are open for them, attempts them with a countdown timer and sees the result at once. The anti-cheat part runs in the student's browser during the exam and sends its events to the Flask server. The main anti-cheat features are listed below.

Feature	What it does
Tab switch / blur detection	Records a violation when the student leaves the exam tab or window
Forced fullscreen	Exam runs in fullscreen and leaving it is recorded
Copy, paste and right-click block	Disables copying, pasting, right-click and common shortcuts
Warnings and auto-submit	Shows a warning each time and submits the exam at the limit (default 3)
Shuffled questions and options	Each student gets a different order
Server-side timer	Start time is saved on the server and checked at submission
One session per student	A second login for the same student is blocked
Webcam snapshots and face check	Photo every 45 seconds; no face or more than one face is recorded

For the report, each violation type gets a weight: tab switch 3, exit fullscreen 3, no face 2, multiple faces 4 and copy-paste attempt 1. The weights are added to make a risk score for the attempt. A score of 0 to 5 is treated as low, 6 to 12 as medium and 13 or more as high. These numbers are only starting values and the teacher can change them. The score is only a guide for the teacher, not a final decision.

Key Features

- Role-based login for Admin, Teacher and Student
- Exam creation with MCQ questions, timer and schedule
- Random question and option order for each student
- Automatic grading and instant result
- Tab, fullscreen and clipboard monitoring
- Violation warnings and automatic submission
- Webcam snapshots and face detection
- Cheat report with timeline and risk score
- Student exam history and result export to CSV
- Responsive pages that also work on tablets and small laptop screens

Methodology

The project will follow an iterative approach where the system is built in small parts and each part is tested before moving to the next. The work is planned in the following steps.

1. Collecting requirements: roles, exam features and anti-cheat rules.
2. Design: architecture, flow charts, use cases and database design.
3. Database and login: models, registration, login and role access.
4. Teacher side: creating and managing exams and questions.
5. Student side: exam page, timer, shuffling and grading.
6. Anti-cheat module: tab, fullscreen and clipboard detection, violation log, auto-submit, then webcam and face detection.
7. Reports: results table, cheat report, risk score and CSV export.
8. Testing and documentation.

For the front end, plain HTML, CSS and JavaScript will be used. The back end will be written in Python with Flask, and SQLite with Flask-SQLAlchemy will store the data. Passwords will be stored as hashes using Werkzeug, and face detection will use face-api.js. Testing will include checking the grading and timer functions, testing the full flow from login to result, and trying to break the anti-cheat on purpose (switching tabs, leaving fullscreen, covering the camera, refreshing the page). A few classmates will also try the system as students and as a teacher. The planned time is about seven weeks:

Week	**Work**
1	Requirements, diagrams and project setup
2	Database, login and dashboards
3	Exam and question management
4	Student exam flow, timer and grading
5	Anti-cheat module and violation log
6	Webcam, face detection and reports
7	Testing, fixing bugs and documentation

System Architecture

The system uses three layers. The presentation layer is what runs in the browser: the student, teacher and admin pages and the anti-cheat script. The application layer is the Flask backend, which handles login, exams, grading, the anti-cheat rules and reports through a simple REST API. The data layer is an SQLite database for records and a folder for the webcam snapshots.

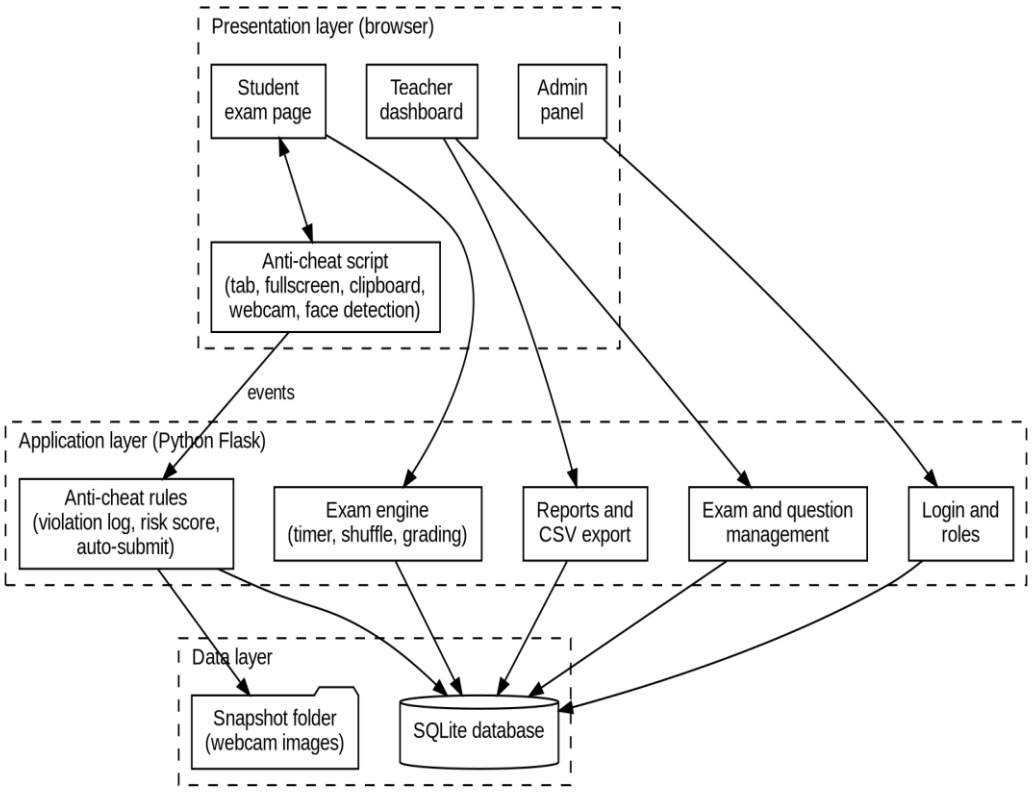


Figure 1: System architecture

System Flow

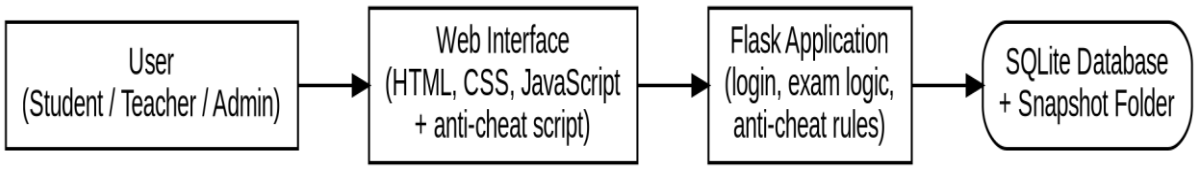


Figure 2: System flow

ER Diagram

Figure 3 shows the main tables. An attempt connects a user with an exam, and the answers, violations and snapshots all belong to an attempt.

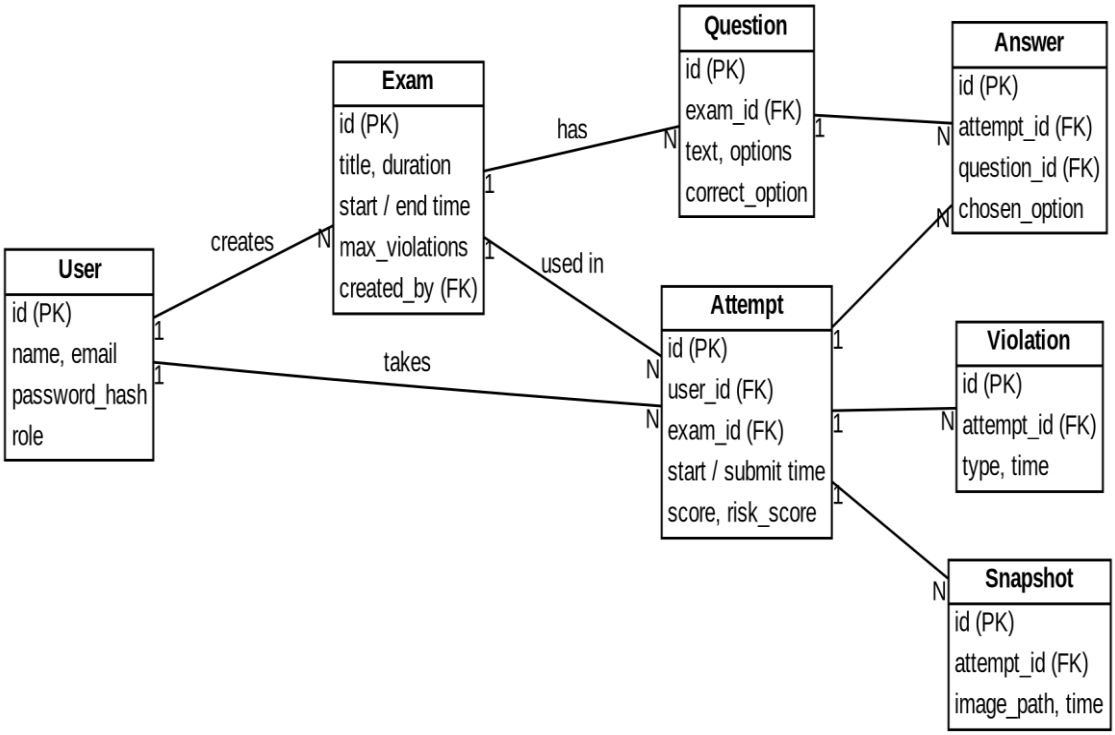


Figure 3: ER diagram

Flow Charts

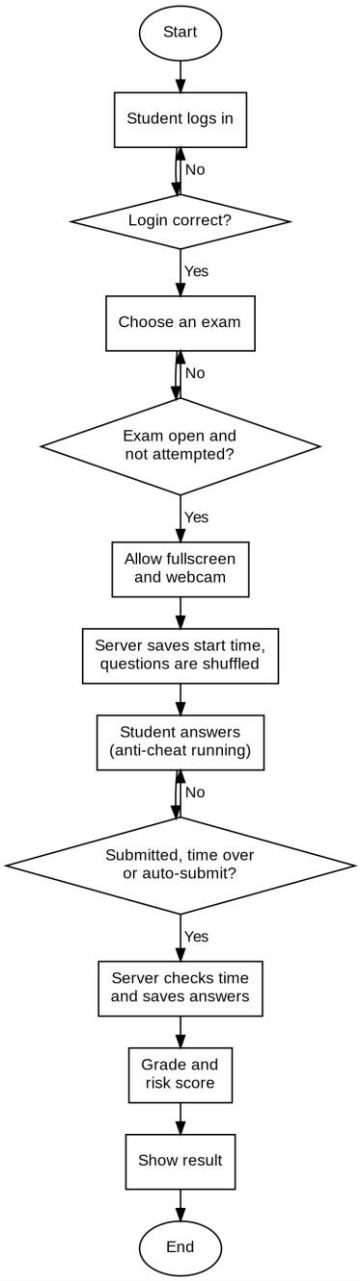


Figure 4: Exam flow chart

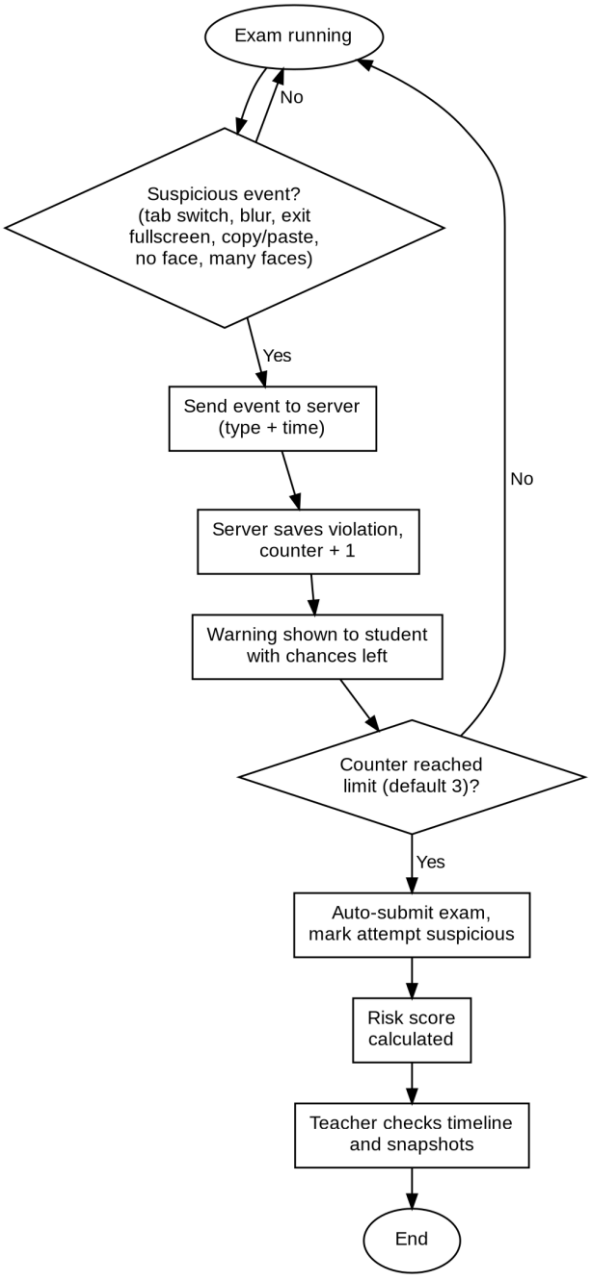


Figure 5: Anti-cheat decision flow

Use Case Diagram

Figure 6 shows what each type of user can do in the system.

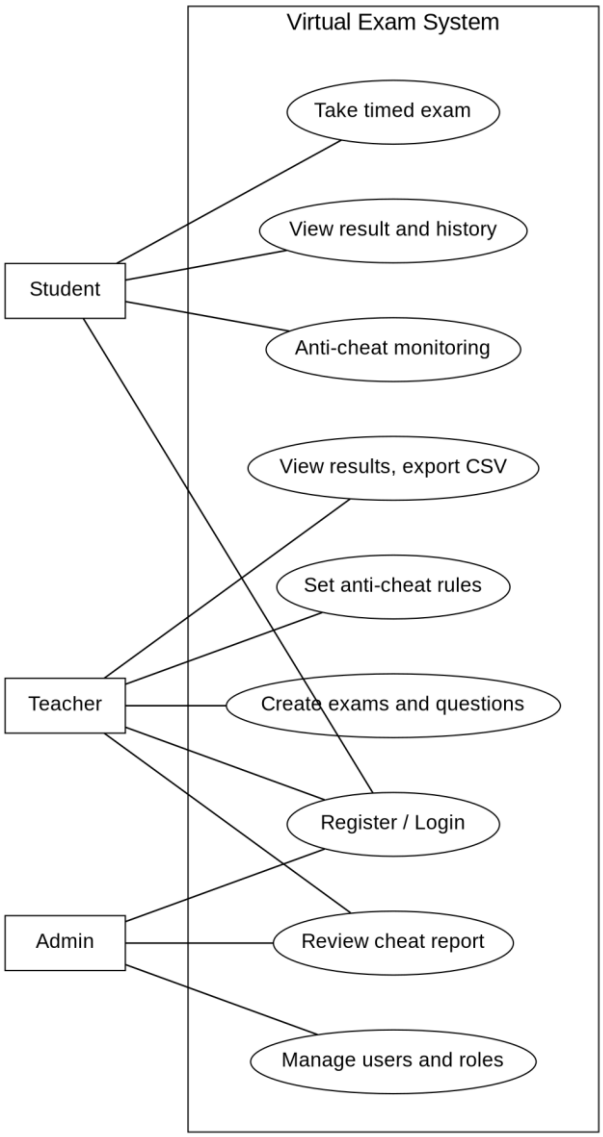


Figure 6: Use case diagram

Limitations

Browser-based anti-cheat can discourage and detect cheating, but it cannot fully prevent it. A student can still use a second phone, take help from a person sitting nearby or run the exam in a virtual machine. There can also be false alarms, for example when a notification takes the focus away, or when the camera is poor or the room is dark. For this reason the system only auto-submits at the limit and leaves the final decision to the teacher. Webcam use also raises privacy questions, so students must be told about it and must give permission, and the snapshots must be stored safely.

Future Work

If time permits, the project can be extended with a question bank, true/false and short-answer questions, PDF export of results, email notification when results are published, and a small analytics page showing which questions were hardest.

Conclusion

The Virtual Exam System with Built-in Anti-Cheat tries to give departments a simple and cheap way to run fair online exams. By combining normal exam features with server-side checking and monitoring in the browser, it helps teachers notice suspicious behaviour and gives them evidence to review. The project is practical for a student to build with HTML, CSS, JavaScript, Flask and SQLite.